SIMATS ENGINEERING ASSESSMENT

TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA51

CO2 ASSESSMENT

Course Outcome Covered:

CO2: Examine public-key cryptographic approaches, including RSA and key exchange, and explain how they support information security.

Assessment focus: Debugging and optimisation

Total Marks: 50

STUDENT DETAILS

Name: Dudekula Mohammad Ilyas

Register Number: 192325017

Code of Conduct

I, ***Dudekula Mohammad Ilyas (Register Number: 192325017)***, certify that this submission represents my work and that I have followed the instructions and academic-integrity requirements applicable to this assessment. I understand that failure to follow these requirements may lead to disciplinary action.

Student signature: **Dudekula Mohammad Ilyas**

QUESTION 3: Debugging and Improving an RSA Cryptosystem

Problem statement: A Python implementation of RSA has difficulty encrypting large messages and may sometimes return incorrect data after decryption.

The tasks are to identify faults in prime generation, key creation and modular exponentiation; improve the implementation with efficient methods such as fast exponentiation and the Chinese Remainder Theorem (CRT); and discuss how RSA key size affects security and speed.

A. Finding Errors and Understanding Their Causes

1. Prime-generation problems

RSA relies on two large prime numbers. If these values are generated incorrectly, the resulting key pair may be weak or unusable. Common problems include:

- Accepting a candidate that is not actually prime.
- Choosing primes that are too small or easy to predict.
- Using a standard pseudo-random generator instead of a cryptographically secure source.
- Allowing p and q to be the same value.
- Using a weak or incorrectly implemented primality test.

Why it matters: weak or unsuitable primes can make the modulus easier to factor and can undermine RSA security.

Improvement: generate candidates with Python's secrets module, test them with a suitable probabilistic test such as Miller–Rabin, ensure p and q differ, and select primes of an appropriate size.

2. Key-generation problems

RSA constructs its public and private keys using the following relationships:

$n = p \times q$

$\varphi(n) = (p - 1)(q - 1)$

$1 < e < \varphi(n)$

$\gcd(e, \varphi(n)) = 1$

$d = e^{-1} \bmod \varphi(n)$

Possible mistakes include choosing a public exponent that is not relatively prime to φ(n), computing the totient incorrectly, using the same prime twice, or calculating the modular inverse incorrectly.

These mistakes can create a private key that does not correctly undo the encryption step. Validate the prime values, compute φ(n) accurately, and calculate d using a reliable modular-inverse operation.

3. Modular-exponentiation problems

RSA uses modular exponentiation for both encryption and decryption:

Encryption: $C = M^e \bmod n$

Decryption: $M = C^d \bmod n$

Common errors are calculating the full power before applying modulo, applying modulo in the wrong place, using incorrect exponents, or passing a message integer that is equal to or larger than n.

The message value must satisfy $0 \leq M < n$. Direct exponentiation can also create extremely large intermediate values and waste time and memory.

Python's three-argument pow() function performs modular exponentiation efficiently:

```
ciphertext = pow(message, e, n)
plaintext = pow(ciphertext, d, n)
```

This approach avoids constructing the complete power and is suitable for large exponents.

B. Optimising the RSA Algorithm

1. Fast modular exponentiation

Repeated squaring reduces the number of multiplications needed to calculate a large power modulo n. For example, calculate 3^{13} mod 7:

$3^2 \bmod 7 = 2$

$3^4 \bmod 7 = 4$

$3^8 \bmod 7 = 2$

$13 = 8 + 4 + 1$

$3^{13} \bmod 7 = (3^8 \times 3^4 \times 3) \bmod 7$

$= (2 \times 4 \times 3) \bmod 7 = 24 \bmod 7 = 3$

Python example:

```
def fast_mod_exp(base, exponent, modulus):
    return pow(base, exponent, modulus)

print(fast_mod_exp(3, 13, 7))
```

Output: 3

Benefits include fewer unnecessary operations, faster handling of large exponents, lower intermediate memory requirements, and improved RSA processing time.

2. Chinese Remainder Theorem (CRT)

CRT can speed up RSA decryption by replacing one large modular-exponentiation calculation with two smaller calculations, one modulo p and one modulo q.

$dP = d \bmod (p - 1)$

$dQ = d \bmod (q - 1)$

$qInv = q^{-1} \bmod p$

$m1 = C^{dP} \bmod p$

$m2 = C^{d}Q \bmod q$

$h = qInv \times (m1 - m2) \bmod p$

$M = m2 + h \times q$

CRT usually makes private-key decryption substantially faster because the two exponentiation operations use smaller moduli.

3. Corrected and optimised Python implementation

The following educational example combines secure random candidate generation, Miller–Rabin testing, key validation, built-in modular exponentiation and CRT-based decryption.

```
import secrets
import math
```

```
def is_prime(n, k=40):
    if n < 2:
        return False
    small_primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    for prime in small_primes:
        if n == prime:
            return True
        if n % prime == 0:
            return False

    d = n - 1
    r = 0
    while d % 2 == 0:
        d //= 2
        r += 1

    for _ in range(k):
        a = secrets.randbelow(n - 3) + 2
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def generate_prime(bits=512):
    while True:
        candidate = secrets.randbits(bits)
        candidate |= (1 << (bits - 1)) | 1
        if is_prime(candidate):
            return candidate

def generate_keys(bits=1024):
    p = generate_prime(bits // 2)
    q = generate_prime(bits // 2)
    while p == q:
        q = generate_prime(bits // 2)

    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    if math.gcd(e, phi) != 1:
        return generate_keys(bits)
    d = pow(e, -1, phi)
    return (e, n), (d, n, p, q)

def encrypt(message, public_key):
    e, n = public_key
    if not 0 <= message < n:
        raise ValueError("Message must be smaller than n")
    return pow(message, e, n)

def decrypt(ciphertext, private_key):
```

```
    d, n, p, q = private_key
    return pow(ciphertext, d, n)

def crt_decrypt(ciphertext, private_key):
    d, n, p, q = private_key
    dp = d % (p - 1)
    dq = d % (q - 1)
    q_inv = pow(q, -1, p)
    m1 = pow(ciphertext, dp, p)
    m2 = pow(ciphertext, dq, q)
    h = (q_inv * (m1 - m2)) % p
    return m2 + h * q

public_key, private_key = generate_keys(1024)
message = 12345
ciphertext = encrypt(message, public_key)
normal_plaintext = decrypt(ciphertext, private_key)
crt_plaintext = crt_decrypt(ciphertext, private_key)

print("Original Message:", message)
print("Encrypted Message:", ciphertext)
print("Normal Decryption:", normal_plaintext)
print("CRT Decryption:", crt_plaintext)
assert normal_plaintext == message
assert crt_plaintext == message
print("RSA encryption and decryption successful!")
```

Expected verification: the original message is 12345, and both normal decryption and CRT decryption return 12345. The ciphertext will vary between runs because new random primes are generated.

Security note: this code is intended for learning and demonstration. Real applications should use a well-reviewed cryptographic library, secure padding such as RSA-OAEP, and recommended key sizes.

4. Encrypting large messages

RSA is not designed to encrypt an arbitrarily large message directly. The integer representing the message must be smaller than the modulus n. For example, if n = 3233, the message value must be below 3233. A larger value cannot be represented correctly by a single RSA operation.

A practical solution is hybrid encryption:

1. Create a random AES session key.
2. Encrypt the data with AES-256-GCM.
3. Protect the AES session key using RSA-OAEP and the recipient's public key.
4. Send the encrypted session key together with the encrypted data.
5. The recipient uses the RSA private key to recover the AES key, then uses AES to decrypt the data.

This design is more efficient for large files because symmetric encryption is much faster for bulk data than RSA.

C. Balancing RSA Key Size, Security and Speed

RSA's security is closely connected to the difficulty of factoring the product of its two prime numbers. Larger keys generally provide stronger protection but require more computation.

RSA key size	Approximate security level	Performance
1024 bits	Not considered secure for modern use	Faster, but unsuitable for new systems
2048 bits	About 112-bit security	A practical balance for many uses

3072 bits	About 128-bit security	Slower than 2048-bit RSA
4096 bits	Higher security margin; estimates vary	More computationally expensive

Analysis

1. 1024-bit keys: they require less processing and may operate faster, but they no longer provide adequate protection for modern applications.

2. 2048-bit keys: they are widely used and offer a useful balance between security and processing cost for many applications.

3. 3072- to 4096-bit keys: they provide a stronger margin against factoring attacks, but encryption, decryption and key generation require more computation.

Technique	**Purpose**	**Expected improvement**
Fast modular exponentiation	Calculate powers efficiently	Reduces calculation time
Chinese Remainder Theorem	Speed up private-key decryption	Reduces decryption time
Secure prime generation	Create dependable RSA keys	Improves key-generation reliability
RSA-OAEP	Add secure encryption padding	Avoids weaknesses of textbook RSA
Hybrid encryption	Handle large data	Improves bulk-data performance

Conclusion

RSA failures commonly arise from weak prime generation, incorrect key parameters, faulty modular arithmetic, or inefficient exponentiation. Secure prime selection, correct modular-inverse calculations, Python's modular pow() function and CRT-based decryption help address these issues.

For large data, hybrid encryption using RSA-OAEP to protect an AES-256-GCM session key is more practical than applying RSA to the entire message. A 2048-bit key is a common balance of security and performance, while larger keys increase the computational cost.

QUESTION 4: Improving and Securing Diffie–Hellman Key Exchange

Problem statement: A Python implementation of Diffie–Hellman has slow calculations and possible security weaknesses.

The required work is to correct shared-secret calculations affected by modular-arithmetic mistakes, improve exponentiation for large prime values, and add protection against man-in-the-middle (MITM) attacks while considering the performance cost.

A. Correcting Shared-Key Generation

1. Overview of Diffie–Hellman

Diffie–Hellman (DH) enables two participants to derive the same secret over a network that may be monitored by others. In this example, the participants are Alice and Bob. They publicly agree on a large prime p and a generator g modulo p. Alice chooses a private value a, while Bob chooses a private value b.

2. Mathematical process

Alice's public value: $A = g^a \bmod p$

Bob's public value: $B = g^b \bmod p$

Alice's shared secret: $K_a = B^a \bmod p$

Bob's shared secret: $K_\beta = A^b \bmod p$

$K_a = (g^b)^a \bmod p = g^{ab} \bmod p$

$K_\beta = (g^a)^b \bmod p = g^{ab} \bmod p$

Both participants therefore calculate the same value: $K_a = K_\beta$.

3. Common mistakes and fixes

Problem	Likely cause	Recommended fix
Shared values do not match	Incorrect modular arithmetic	Use correct modular exponentiation
Alice and Bob derive different keys	Wrong public/private-key calculation	Check the DH equations and values
Private values are weak	Predictable random-number source	Use Python's secrets module
Invalid prime modulus	Unsuitable parameters	Validate approved parameters
Incorrect generator	Invalid generator selection	Use approved DH parameters
Invalid private exponent	Missing range checks	Restrict private values to an appropriate range
MITM exposure	Public keys are not authenticated	Use signatures or an authenticated key exchange

4. Worked example

Given p = 23, g = 5, Alice's private key a = 6, and Bob's private key b = 15:

Step 1 — Alice's public key:

$A = 5^6 \bmod 23 = 15625 \bmod 23 = 8$

Step 2 — Bob's public key:

$5^2 \bmod 23 = 2$; $5^4 \bmod 23 = 4$; $5^8 \bmod 23 = 16$.

Because $15 = 8 + 4 + 2 + 1$:

$5^{15} \bmod 23 = (16 \times 4 \times 2 \times 5) \bmod 23 = 640 \bmod 23 = 19$

Therefore, B = 19.

Step 3 — Alice derives the shared value:

$K_a = 19^6 \bmod 23$. Since $19^2 \bmod 23 = 16$ and $19^4 \bmod 23 = 3$, $K_a = (3 \times 16) \bmod 23 = 2$.

Step 4 — Bob derives the shared value:

$K_\beta = 8^{15} \bmod 23$. Using $8^2 \bmod 23 = 18$, $8^4 \bmod 23 = 2$ and $8^8 \bmod 23 = 4$:

$K_\beta = (4 \times 2 \times 18 \times 8) \bmod 23 = 1152 \bmod 23 = 2$.

Verification: $K_a = K_\beta = 2$. Each party obtains the same secret without sending its private value across the network.

B. Improving Exponentiation for Large Prime Values

1. Fast modular exponentiation

DH uses modular exponentiation to generate public values and shared secrets. Calculating a complete power first can be slow and can consume unnecessary memory. Python's three-argument pow() function performs the operation efficiently.

```
p = 23
g = 5
a = 6
A = pow(g, a, p)
print("Alice Public Key:", A)
```

Output: Alice Public Key: 8

The function calculates the result modulo p during exponentiation, avoiding the need to store a huge intermediate power.

2. Secure private-key generation

Ordinary random-number functions are not appropriate for private cryptographic values because their output may be predictable.

Less suitable example:

```
import random
private_key = random.randint(1, 100)
```

Improved example:

```
import secrets
private_key = secrets.randbelow(p - 3) + 2
```

This generates a cryptographically strong value in the range from 2 to p − 2 for the educational parameter set.

3. Optimisation methods

Method	Purpose	Benefit
Fast modular exponentiation	Calculate modular powers efficiently	Faster key calculations
Secure random generation	Choose unpredictable private values	Improves secrecy

Parameter validation	Check the prime and generator	Avoids invalid operations
Reuse validated parameters	Avoid repeated checks of trusted parameters	Reduces overhead
Key derivation	Turn the shared secret into usable keys	Supports safer communication

4. Corrected Diffie–Hellman demonstration

```
import secrets

# Small values are used only to demonstrate the calculation.
p = 23
g = 5

if not 2 <= g <= p - 2:
    raise ValueError("Invalid generator range")

alice_private = secrets.randbelow(p - 3) + 2
bob_private = secrets.randbelow(p - 3) + 2

alice_public = pow(g, alice_private, p)
bob_public = pow(g, bob_private, p)

alice_shared = pow(bob_public, alice_private, p)
bob_shared = pow(alice_public, bob_private, p)

print("Alice Public Key:", alice_public)
print("Bob Public Key:", bob_public)
print("Alice Shared Key:", alice_shared)
print("Bob Shared Key:", bob_shared)

if alice_shared == bob_shared:
    print("Diffie-Hellman Key Exchange Successful!")
else:
    print("Key Exchange Failed!")

assert alice_shared == bob_shared
```

The public values and shared secret vary between runs because the private values are generated randomly. The key condition is that both shared-secret calculations match.

Important limitation: p = 23 is far too small for real security and is used only to illustrate the mathematics. Production systems should use standardised, sufficiently large DH parameters and validate received public values.

C. Preventing MITM Attacks and Measuring Performance

1. How a man-in-the-middle attack works

A MITM attacker may intercept public-key messages between Alice and Bob and replace them with different values. Alice may then establish a secret with the attacker, while Bob establishes a separate secret with the same attacker. The attacker could consequently read or alter messages while each participant mistakenly believes the connection is directly with the other.

2. Methods to reduce MITM risk

Digital signatures

Each participant signs its DH public value using a private signing key. The other participant checks the signature with a trusted verification key. This makes unauthorised replacement of public values detectable when the signing keys are correctly authenticated.

Digital certificates

Certificates issued by trusted Certificate Authorities can bind public keys to identities. Checking certificates helps prevent an attacker from impersonating either participant.

Message Authentication Code (MAC)

A MAC, such as HMAC, verifies message authenticity and integrity when both parties already possess a shared authentication secret. An outsider without that secret should not be able to produce a valid tag.

Secure key derivation

A Key Derivation Function such as HKDF should be applied to the DH shared secret to produce suitable encryption keys. The raw shared secret should not be used directly as an AES key.

3. Example of an authenticated exchange

The following educational example illustrates HMAC-based checking using a pre-shared authentication key. It is not a complete production protocol.

```
import secrets
import hashlib
import hmac

# Small demonstration parameters only
p = 23
g = 5
authentication_key = b"shared-authentication-secret"

alice_private = secrets.randbelow(p - 3) + 2
bob_private = secrets.randbelow(p - 3) + 2

alice_public = pow(g, alice_private, p)
bob_public = pow(g, bob_private, p)

transcript = f"Alice:{alice_public}|Bob:{bob_public}".encode()
alice_mac = hmac.new(authentication_key, transcript, hashlib.sha256).hexdigest()
bob_mac = hmac.new(authentication_key, transcript, hashlib.sha256).hexdigest()

alice_verified = hmac.compare_digest(alice_mac, bob_mac)
bob_verified = hmac.compare_digest(bob_mac, alice_mac)

if not alice_verified or not bob_verified:
    raise ValueError("Authentication failed!")

alice_shared = pow(bob_public, alice_private, p)
bob_shared = pow(alice_public, bob_private, p)

if alice_shared == bob_shared:
    print("Authentication Successful!")
    print("Shared Secret:", alice_shared)
```

```
        print("Secure Key Exchange Completed!")
    else:
        print("Key Exchange Failed!")
```

How it works: Alice and Bob create their key pairs, place the public values in a transcript, compute HMAC values using the previously shared authentication secret, and compare the results. If authentication succeeds, they calculate and compare the DH shared secret.

Security limitation: because the example assumes a shared authentication secret and uses tiny public parameters, it is for demonstration only. A real deployment should use a standard authenticated protocol, such as mutually authenticated TLS or a properly designed authenticated key exchange. The authentication should cover the full transcript, identities and session context, and HKDF should derive separate keys for separate purposes.

4. Performance cost of security improvements

Technique	Security benefit	Performance effect
Digital signatures	Confirms ownership of a public key	Adds signing and verification work
Digital certificates	Checks participant identity	Requires certificate validation
HMAC	Checks authenticity and integrity	Usually low overhead
HKDF	Derives secure session keys	Adds a small amount of computation
Fast modular exponentiation	Efficient DH calculations	Reduces calculation time

5. Before-and-after comparison

Area	Basic implementation	Improved implementation
Modular exponentiation	May calculate ordinary powers	Uses built-in modular pow()
Private-key generation	May use predictable randomness	Uses cryptographically secure randomness
Public-value validation	May be omitted	Validates inputs and parameters
Authentication	Not included	Adds HMAC or another authenticated protocol
MITM resistance	Vulnerable to public-key replacement	Improved through authenticated exchange
Memory use	May create large intermediate values	Avoids unnecessary intermediate values
Overall security	Weak or incomplete	Improved when standard parameters and protocols are used

Performance discussion

Fast modular exponentiation reduces the time spent calculating large powers. Secure random generation adds a small overhead but is necessary for private-key security. HMAC requires additional hashing, which is generally inexpensive. Digital signatures and certificate checks add more processing, while HKDF requires a modest amount of extra work. Overall, these operations may slightly increase execution time but provide important security benefits.

Conclusion

Incorrect Diffie–Hellman shared values can result from mistakes in modular arithmetic, key calculations or parameter validation. Using Python's built-in pow() function, secure private-value generation and suitable parameter checks improves correctness and efficiency.

Unauthenticated Diffie–Hellman does not confirm who owns an exchanged public key, so it can be exposed to MITM attacks. Digital signatures, trusted certificates or a correctly implemented HMAC-based protocol can authenticate the exchange. Although authentication introduces some extra processing, it substantially improves communication security.

Therefore, an optimised and properly authenticated Diffie–Hellman design offers more reliable calculations, better performance and stronger resistance to public-key substitution attacks.